CH#05:-

Error Detection:-

CRC: r+1 pattern → generator (G)
d+r → should be divisible by G
R = remainder $\frac{D \cdot 2^r}{G}$

XOR

1	0	0
0	1	1
1	0	1
1	1	0

Random Access Protocols: R-one node, R_M for more, Decentralized, Inexpensive

i-Channel Partioining: a node is given a slot in each frame. R/N bps.
i-TDM, ii-FDM, iii-CDMA: assign Codes to each node.

ii- Random Access: R bps, random before retransmit
- Slotted Aloha:
  slot size = L/R seconds
  - Collision Detected by All nodes
  - Efficiency = $Np(1-p)^{N-1}$
    $= 1/e = 0.37$
- ALOHA:
  - no slots.
  - Retransmit ASAP - $1/2e$
- CSMA or CSMA/CD
  - propogation delay can cause collision. (ESMD)
    ↓
    fixed in CSMA/CD
  - Binary exponential Backoff: (used to decide delay)
    K.512 bit times i.e bit time = time to send a bit

$K = [0, 1, 2, 3, \cdots, 2^{n-1}]$, n = no. of collisions

$$\text{Efficiency} = \frac{1}{1 + 5d_{prop}/d_{trans}}$$

iii- Taking Turns:
- Polling: master polls all nodes in round-robin fashion.
  - Polling Delay
- Token Passing:
  - Special Purpose frame - Token

Switched Local Area Network:

- MAC: link layer address
  - 6-bytes long
  - Provided by IEEE, first 24-bit fixed
  - FF-FF-FF-FF-FF-FF is Broadcast MAC

- ARP:
  - Every host has this ARP table in a LAN
  - ARP Query & response Packet
  - link Layer protocol
- Ethernet:

preamble	dest addr	Src addr	type	data (payload)	CRC

preamble → sync sender, receiver clocks

  - MAC protocol: CSMA/CD with binary backOff
- Switches:
  - Self learning, plug-n-play
  - Switch Table:

MAC addr	interface	time Stamp

- Data Centre:
  - Load Balancer:
  - Increase reliability vs redundancy

CH#4:

Dijkstra's:

Step	N'	D(v) p(v)

Distance Vector Algo:

Hierarchical Routing:
- ASes
  - Same AS: - Same protocol - Intra AS
  - Different AS: Inter AS: → both make router's forwarding table
    - Get info from neighbour AS's.
    - Tell this reachability info to all the routers in Intra-AS
    - Hot Potato Routing: Send towards the closest of Two.

Intra-AS: - RIP:
- Distance Vector Algo

Subnet	hops	next

hops → not > 15

- managed by app route-d (daemon)
- UDP

- OSPF: Hierarchy of routers: Boundary, Backbone, region (area border)
  route Selection: policy, AS-PATH, Hot-Potato,

CH#01:

End-to-End Delay = $d_{trans} + d_{prop} + d_{queue} + d_{proc}$

$d_{trans} = \frac{L}{R}$, $d_{prop} = d/s$

bandwidth. delay product = $R . \frac{L}{R}$

Traffic Intensity = $\frac{La}{R}$ → avg pckt arrival rate

- ~0 : small queueing delay
- <=1 : Some increase in // //
- >1 : more work than can be processed, delay can reach infinity

Throughput : (bits/time) = R

Area of networks:
personal = 1m LAN = 1km, MAN = 10 km, WAN = 1000 km, Internet = 10,000 km

mesh Topology: no. of Links = $\frac{n(n-1)}{2}$

Frame size = (no. of channels) x (size of one slot)

frame rate = bit rate / frame size

bit rate = (no. of bits in a sec of a channel) x (no. of channels)

bit Duration = $\frac{1}{\text{bit rate}}$

CH#02:

non-persistent HTTP + non pipelining = 2RTT + FTT → file Transmission time.

// // // + pipelined = 2RTT + 2(n(RTT)) + n(FTT)

(2RTT: initial connection; n: no. of objects to get)

persisted HTTP + no-pipelining = RTT + n(RTT) + n(FTT)

(RTT: for connection; n: no. of obj's)

// // + pipelining = RTT + n(RTT/no. of parallel connections allowed) + n(FTT/no. of parallel con. allowed).

~~Cookies~~ Web Caching:

Traffic Intensity = $\frac{La}{R}$

Total Response Time = LAN Delay + access Delay + Internet Delay

Hit Rate = Requests fullfilled by cache. normally (0.2 to 0.7).

avg Delay after caching = Delay for response (of cache + of Internet)

CH # 03:

Sender Utilization = $\frac{L/R}{RTT + L/R}$

// // in pipelining = $\frac{n(L/R)}{RTT + L/R}$ → number of packets

GBN: rcv_pkt() → sendAck()
pkt_timedout() → resend()
Pkt_lost() → sendDuplicateAck()
Discard out of order

SR:
send ack for each rcved pkt.
Buffer out of Order.

window size = half of seq. no. space.

TCP:
Seq.no = ~~ett~~ first byte of current pkt
ack.no = next byte that the ~~et~~ client should send.

RTT: Timeout Interval = Estimated Timeout + 4(Dev RTT)

Est. RTT = $(1-\alpha)$*Est. RTT + α*SampleRTT

Dev-RTT = $(1-\beta)$*Dev.RTT + β*(SampleRTT − EstimatedRTT)

$\alpha = 0.125$, $\beta = 0.25$

Flow Control: rwnd = RcvBuffer − [LastByteRcvd − LastByteRead]
For sender = rwnd >= lastByteSent − LastByteAcked.

Congestion Control:

min(rwnd, cwnd) ≥ LastByteSent − lastByteAcked

rate of sending ≈ $\frac{cwnd}{RTT}$

Algo: cwnd = 1 MSS at the start
ssthresh = cwnd/2

Tahoe: Timeout or 3 dup Acks: cwnd = 1, ssthresh = cwnd/2

Reno: 3 dup Acks: ~~cwnd is cut to half, then congestion avoidance~~
cwnd in ssthresh + 3, fast recovery until some Ack received. Then CA

Timeout: cwnd = 1, ssthresh = cwnd/2